

Libft

İlk kendi kütüphanen

Özet: Bu proje, programların için sayısız genel amaçlı fonksiyon içerecek bir `C` kütüphanesi yazmayı kapsar.

Versiyon: 19.3

(Türkçe çeviri — teknik terimler orijinal haliyle korunmuştur)

İçindekiler

I. Giriş

II. Ortak Talimatlar

III. AI Talimatları

IV. Mandatory Part (Zorunlu Kısım)

- IV.1 Teknik hususlar
- IV.2 Part 1 - Libc functions
- IV.3 Part 2 - Additional functions
- IV.4 Part 3 - linked list

V. Readme Gereksinimleri

VI. Submission and peer-evaluation

Bölüm I — Giriş

C programlama, çok kullanışlı standart fonksiyonlara erişim olmadan oldukça yorucu olabilir. Bu proje, bu fonksiyonların nasıl çalıştığını, onları kendin implemente ederek anlamana ve etkili bir şekilde kullanmayı öğrenmene yardımcı olmayı amaçlar. Gelecekteki C okul ödevlerin için değerli olacak kendi kütüphaneni oluşturacaksın.

Bölüm II — Ortak Talimatlar

- Projen `C` ile yazılmış olmalıdır.
- Projen `Norm` 'a uygun yazılmış olmalıdır. Bonus dosyaların/fonksiyonların varsa, bunlar da norm kontrolüne dahildir ve bir norm hatası varsa `0` alırsın.
- Fonksiyonların beklenmedik şekilde sonlanmamalıdır (segmentation fault, bus error, double free vb.), undefined behavior hariç. Bu gerçekleşirse projen çalışmıyor sayılır ve değerlendirmede `0` alır.
- Heap'te tahsis edilen tüm bellek gerektiğinde düzgünce serbest bırakılmalıdır. Memory leak'lere (bellek sızıntısı) tolerans gösterilmez.
- Subject gerektiriyorsa, kaynak dosyalarını istenen çıktıya `-Wall`, `-Wextra` ve `-Werror` bayraklarıyla, `cc` kullanarak derleyen bir `Makefile` teslim etmelisin. Ayrıca `Makefile` 'in gereksiz yeniden linkleme (relinking) yapmaması gerekir.
- `Makefile` 'in en azından şu kuralları içermesi gerekir: `$(NAME)`, `all`, `clean`, `fclean` ve `re`.
- Proje için bonus teslim etmek istiyorsan, `Makefile` 'na bir `bonus` kuralı eklemelisin; bu kural projenin ana kısmında izin verilmeyen tüm header'ları, kütüphaneleri veya fonksiyonları ekleyecektir. Subject aksini belirtmedikçe, bonuslar `_bonus.{c/h}` dosyalarına yerleştirilmelidir. Mandatory (zorunlu) ve bonus kısımların değerlendirmesi ayrı ayrı yapılır.
- Proje `libft` 'ini kullanmana izin veriyorsa, kaynak dosyalarını ve ilişkili `Makefile` 'ını bir `libft` klasörüne kopyalamalısın. Projenin `Makefile` 'ı, kütüphaneyi kendi `Makefile` 'ını kullanarak derlemeli, ardından projeyi derlemelidir.
- Proje için test programları oluşturmanı tavsiye ederiz; bu iş teslim edilmesi gerekmez ve not verilmez. Bu, hem kendi çalışmanı hem de akranlarının çalışmasını kolayca test etme fırsatı verir. Bu testleri savunma (defence) sırasında özellikle faydalı bulacaksın. Nitekim savunma sırasında, kendi testlerini ve/veya değerlendirdiğin akranın testlerini kullanmakta özgürsün.
- Çalışmanı atanan Git deposuna (repository) gönder. Yalnızca Git deposundaki çalışma değerlendirilecektir. Deepthought çalışmanı değerlendirmekle görevlendirilmişse, bu akran değerlendirmelerinden (peer-evaluation) sonra gerçekleşir. Deepthought'un değerlendirmesi sırasında çalışmanın herhangi bir bölümünde bir hata oluşursa, değerlendirme durur.

Bölüm III — AI Talimatları

• Bağlam

Bu proje, 42 eğitiminin temel yapı taşlarını keşfetmene yardımcı olmak için tasarlanmıştır.

Kilit bilgi ve becerileri doğru şekilde yerleştirmek için, AI araçlarını ve desteğini kullanma konusunda düşünceli bir yaklaşım benimsemek esastır.

Gerçek temel öğrenme, meydan okuma (challenge), tekrar (repetition) ve akranlarla öğrenme (peer-learning) alışverişleri yoluyla gerçek bir entelektüel çaba gerektirir.

AI konusundaki duruşumuza — bir öğrenme aracı olarak, 42 eğitiminin bir parçası olarak ve iş piyasasında bir beklenti olarak — dair daha kapsamlı bir genel bakış için lütfen intranet'teki özel FAQ'a başvurun.

• Ana mesaj

- Kısayol olmadan sağlam temeller inşa et.
- Teknik ve güç becerilerini gerçekten geliştir.
- Gerçek akran öğrenimini deneyimle, yeni problemleri nasıl öğreneceğini ve çözeceğini öğrenmeye başla.
- Öğrenme yolculuğu, sonuçtan daha önemlidir.
- AI ile ilişkili riskleri öğren ve yaygın tuzaklardan kaçınmak için etkili kontrol pratikleri ve karşı önlemler geliştir.

• Öğrenci kuralları

- Sana atanan görevlere, özellikle AI'ya başvurmadan önce, akıl yürütme (reasoning) uygulamalısın.
- AI'dan doğrudan cevaplar istememelisin.
- 42'nin AI konusundaki küresel yaklaşımı hakkında bilgi edinmelisin.

• Aşama çıktıları

Bu temel aşama içinde, şu çıktıları elde edeceksin:

- Uygun teknik ve kodlama temellerini kazanmak.
- Bu aşamada AI'nın neden ve nasıl tehlikeli olabileceğini bilmek.

• Yorumlar ve örnek

- Evet, AI'nın var olduğunu biliyoruz — ve evet, projelerini çözebilir. Ama sen burada öğrenmek için varsın, AI'nın öğrendiğini kanıtlamak için değil. Verilen problemi AI'nın çözebildiğini göstermek için zamanını (veya bizimkini) boşa harcamaz.
- 42'de öğrenmek, cevabı bilmekle ilgili değildir — bir cevap bulma yeteneğini geliştirmekle ilgilidir. AI sana cevabı doğrudan verir, ama bu senin kendi akıl yürütmeni inşa etmeni engeller. Ve akıl yürütme zaman, çaba gerektirir ve başarısızlığı da içerir. Başarıya giden yol kolay olmak zorunda değildir.
- Sınavlar sırasında AI'nın mevcut olmadığını unutma — internet yok, akıllı telefon yok vb. Öğrenme sürecinde AI'ya çok fazla güvenip güvenmediğini hızlıca fark edeceksin.
- Akran öğrenimi seni farklı fikirlere ve yaklaşımlara maruz bırakır, kişilerarası becerilerini ve farklı/ayrışık (divergent) düşünme yeteneğini geliştirir. Bu, sadece bir bot ile sohbet etmekten çok daha değerlidir. Bu yüzden çekinme — konuş, sorular sor ve birlikte öğrenin!
- Evet, AI müfredatın bir parçası olacak — hem bir öğrenme aracı hem de kendi başına bir konu olarak. Hatta kendi AI yazılımını inşa etme şansın olacak. Gececeğin crescendo (kademeli artan) yaklaşımımız hakkında daha fazla bilgi edinmek için intranette mevcut dokümantasyona bakabilirsin.

✓ İyi pratik:

Yeni bir kavramda takılıyorum. Yakınımdaki birine buna nasıl yaklaştığımı soruyorum. 10 dakika konuşuyoruz — ve birden kafamda oturuyor. Anlıyorum.

✗ Kötü pratik:

Gizlice AI kullanıyorum, doğru görünen bir kod kopyalıyorum. Akran değerlendirmesi sırasında hiçbir şeyi açıklayamıyorum. Kalıyorum (başarısız oluyorum). Sınavda — AI yok — yine takılıyorum. Kalıyorum.

Bölüm IV — Mandatory Part (Zorunlu Kısım)

Program Adı	libft.a
Teslim Edilecek Dosyalar	Makefile, libft.h, ft_*.c
Makefile	NAME, all, clean, fclean, re
External Function	Aşağıda detaylandırılmıştır
Libft izinli mi	n/a
Açıklama	Kendi kütüphaneni oluştur: müfredatın için kullanışlı bir araç olacak fonksiyonlar koleksiyonu.

IV.1 Teknik hususlar

- Global değişken tanımlamak kesinlikle yasaktır.
- Daha karmaşık bir fonksiyonu parçalara ayırmak için yardımcı fonksiyonlara ihtiyacın varsa, bunları ilgili dosyaya kapsamını sınırlamak için `static` fonksiyonlar olarak tanımla.
- Tüm dosyalar deponun (repository) kök dizininde (root) bulunmalıdır.
- Kullanılmayan dosyaların teslim edilmesine izin verilmez.
- Her `.c` dosyası şu bayraklarla derlenmelidir: `-Wall -Wextra -Werror`.
- Kütüphaneni oluşturmak için `ar` komutunu kullanmalısın. `libtool` kullanımı kesinlikle yasaktır.
- `libft.a` 'n deponun kök dizininde oluşturulmalıdır.

IV.2 Part 1 — Libc functions (Libc fonksiyonları)

Başlangıç olarak, `libc` 'den bir dizi fonksiyonu yeniden implemente etmelisin. Versiyonun, orijinalerle aynı prototiplere ve davranışlara sahip olacak, `man page` 'lerindeki tanımlarına kesinlikle bağlı kalacaktır. Tek fark, `'ft_'` öneki ile başlaması gereken isimleri olacaktır. Örneğin, `strlen`, `ft_strlen` olur.

Yeniden implemente etmen gereken bazı fonksiyon prototipleri `'restrict'` niteleyicisini (qualifier) kullanır. Bu anahtar kelime `C99` standardının bir parçasıdır. Bu nedenle, bunu kendi prototiplerine dahil etmen ve kodunu `-std=c99` bayrağıyla derlemen yasaktır.

Aşağıdaki orijinalleri implemente eden kendi fonksiyonlarını yazmalısın. Bunlar hiçbir external (dış) fonksiyona bağımlı olmamalıdır.

Karakter sınıflandırma fonksiyonları için (`isalpha` , `isdigit` , `isalnum` , `isascii` , `isprint`), dönüş değeri şöyle olmalıdır:

- Test edilen sınıfla eşleşiyorsa **1**
- Eşleşmiyorsa **0**

- | | | |
|------------------------|------------------------|------------------------|
| • <code>isalpha</code> | • <code>bzero</code> | • <code>strchr</code> |
| • <code>isdigit</code> | • <code>memcpy</code> | • <code>strrchr</code> |
| • <code>isalnum</code> | • <code>memmove</code> | • <code>strncmp</code> |
| • <code>isascii</code> | • <code>strncpy</code> | • <code>memchr</code> |
| • <code>isprint</code> | • <code>strlcat</code> | • <code>memcmp</code> |
| • <code>strlen</code> | • <code>toupper</code> | • <code>strnstr</code> |
| • <code>memset</code> | • <code>tolower</code> | • <code>atoi</code> |

Aşağıdaki iki fonksiyonu implemente etmek için `malloc()` kullanacaksın:

- `calloc`
- `strdup`

İşletim sistemine bağlı olarak, `'calloc'` fonksiyonunun davranışı `man page` açıklamasından farklı olabilir. Şu kurala uymalısın: `nmemb` veya `size 0` ise, `calloc()` `free()` 'ye başarıyla geçirilebilecek benzersiz (unique) bir pointer değeri döndürür.

Yeniden implemente etmen gereken `strncpy` , `strlcat` ve `bzero` gibi bazı fonksiyonlar, varsayılan olarak GNU C Library (`glibc`) içine dahil değildir. Bunları sistem standardına karşı test etmek için `<bsd/string.h>` dahil etmen ve `-lbsd` bayrağıyla derlemen gerekebilir. Bu davranış `glibc` sistemlerine özgüdür. Meraklıysan, `glibc` ile BSD `libc` arasındaki farkları keşfetme fırsatını değerlendir.

IV.3 Part 2 — Additional functions (Ek fonksiyonlar)

Bu ikinci kısımda, `libc` 'de bulunmayan ya da farklı bir formda bulunan bir dizi fonksiyon geliştirmelisin.

Part 1'deki bazı fonksiyonlar, aşağıdaki fonksiyonları implemente etmek için faydalı olabilir.

ft_substr

Prototip	<code>char *ft_substr(char const *s, unsigned int start, size_t len);</code>
Parametreler	<code>s</code> : alt dizenin (substring) oluşturulacağı orijinal dize. <code>start</code> : alt dizenin <code>s</code> içindeki başlangıç indeksi. <code>len</code> : alt dizenin maksimum uzunluğu.
Dönüş Değeri	Alt dize. Tahsis (allocation) başarısız olursa <code>NULL</code> .
External Function	<code>malloc</code>
Açıklama	Bellek tahsis eder (<code>malloc(3)</code> kullanarak) ve <code>s</code> dizesinden bir alt dize döndürür. Alt dize, <code>start</code> indeksinden başlar ve maksimum <code>len</code> uzunluğuna sahiptir.

ft_strjoin

Prototip	<code>char *ft_strjoin(char const *s1, char const *s2);</code>
Parametreler	<code>s1</code> : önek (prefix) dizesi. <code>s2</code> : sonek (suffix) dizesi.
Dönüş Değeri	Yeni dize. Tahsis başarısız olursa <code>NULL</code> .
External Function	<code>malloc</code>
Açıklama	Bellek tahsis eder (<code>malloc(3)</code> kullanarak) ve <code>s1</code> ile <code>s2</code> 'yi birleştirmenin (concatenating) sonucu olan yeni bir dize döndürür.

ft_strtrim

Prototip	<code>char *ft_strtrim(char const *s1, char const *set);</code>
Parametreler	<code>s1</code> : kırpılacak (trimmed) dize. <code>set</code> : kaldırılacak karakter kümesini içeren dize.
Dönüş Değeri	Kırpılmış dize. Tahsis başarısız olursa <code>NULL</code> .
External Function	<code>malloc</code>
Açıklama	Bellek tahsis eder (<code>malloc(3)</code> kullanarak) ve <code>s1</code> 'in, başından ve sonundan <code>set</code> 'teki karakterler kaldırılmış bir kopyasını döndürür.

ft_split

Prototip	<code>char **ft_split(char const *s, char c);</code>
Parametreler	<code>s</code> : bölünecek (split) dize. <code>c</code> : ayırıcı (delimiter) karakter.
Dönüş Değeri	Bölme işleminden elde edilen yeni dizelerin dizisi (array). Herhangi bir tahsis başarısız olursa <code>NULL</code> . Döndürülen yapı şu şekilde serbest bırakılacaktır: 1) array içindeki her dize üzerinde <code>free()</code> ; 2) array'in kendisinde <code>free()</code> .
External Function	<code>malloc</code> , <code>free</code>
Açıklama	Bellek tahsis eder (<code>malloc(3)</code> kullanarak) ve <code>s</code> 'nin <code>c</code> karakteri ayırıcı olarak kullanılarak bölünmesiyle elde edilen bir dize dizisi döndürür. Döndürülen dizideki her dize bağımsız olarak tahsis edilir. Pointer dizisinin kendisi de dinamik olarak tahsis edilir. Döndürülen dizi <code>NULL</code> ile sonlandırılmış olmalıdır.

ft_itoa

Prototip	<code>char *ft_itoa(int n);</code>
Parametreler	<code>n</code> : dönüştürülecek integer.
Dönüş Değeri	Integer'ı temsil eden dize. Tahsis başarısız olursa <code>NULL</code> .

External Function	<code>malloc</code>
Açıklama	Bellek tahsis eder (<code>malloc(3)</code> kullanarak) ve argüman olarak alınan integer'ı temsil eden bir dize döndürür. Negatif sayılar işlenmelidir (handled).

ft_strmapi

Prototip	<code>char *ft_strmapi(char const *s, char (*f)(unsigned int, char));</code>
Parametreler	<code>s</code> : üzerinde iterasyon yapılacak dize. <code>f</code> : her karaktere uygulanacak fonksiyon.
Dönüş Değeri	<code>f</code> 'nin ardışık uygulamalarından oluşturulan dize. Tahsis başarısız olursa <code>NULL</code> döner.
External Function	<code>malloc</code>
Açıklama	<code>f</code> fonksiyonunu <code>s</code> dizesinin her karakterine uygular; ilk argüman olarak indeksini, ikinci argüman olarak karakterin kendisini geçirir. <code>f</code> 'nin ardışık uygulamalarından elde edilen sonuçları saklamak için yeni bir dize oluşturulur (<code>malloc(3)</code> kullanarak).

ft_striteri

Prototip	<code>void ft_striteri(char *s, void (*f)(unsigned int, char*));</code>
Parametreler	<code>s</code> : üzerinde iterasyon yapılacak dize. <code>f</code> : her karaktere uygulanacak fonksiyon.
Dönüş Değeri	Yok.
External Function	Yok.
Açıklama	<code>f</code> fonksiyonunu argüman olarak verilen dizenin her karakterine uygular; ilk argüman olarak indeksini geçirir. Her karakter, gerekirse değiştirilebilmesi için <code>f</code> 'ye adresi (address) ile geçirilir.

ft_putchar_fd

Prototip	<code>void ft_putchar_fd(char c, int fd);</code>
Parametreler	<code>c</code> : çıktı verilecek karakter. <code>fd</code> : yazılacak dosya tanımlayıcısı (file descriptor).
Dönüş Değeri	Yok.
External Function	<code>write</code>
Açıklama	<code>c</code> karakterini belirtilen dosya tanımlayıcısına çıktı verir.

ft_putstr_fd

Prototip	<code>void ft_putstr_fd(char *s, int fd);</code>
Parametreler	<code>s</code> : çıktı verilecek dize. <code>fd</code> : yazılacak dosya tanımlayıcısı.
Dönüş Değeri	Yok.
External Function	<code>write</code>
Açıklama	<code>s</code> dizesini belirtilen dosya tanımlayıcısına çıktı verir.

ft_putendl_fd

Prototip	<code>void ft_putendl_fd(char *s, int fd);</code>
Parametreler	<code>s</code> : çıktı verilecek dize. <code>fd</code> : yazılacak dosya tanımlayıcısı.
Dönüş Değeri	Yok.
External Function	<code>write</code>
Açıklama	<code>s</code> dizesini, ardından bir satır sonu (newline) ile belirtilen dosya tanımlayıcısına çıktı verir.

ft_putnbr_fd

Prototip	<code>void ft_putnbr_fd(int n, int fd);</code>
Parametreler	<code>n</code> : çıktı verilecek integer. <code>fd</code> : yazılacak dosya tanımlayıcısı.
Dönüş Değeri	Yok.
External Function	<code>write</code>
Açıklama	<code>n</code> integer'ını belirtilen dosya tanımlayıcısına çıktı verir.

IV.4 Part 3 — linked list (bağlı liste)

Bellek ve dize (string) manipülasyon fonksiyonları faydalıdır. Ama yakında listeleri (list) manipüle etmenin daha da faydalı olduğunu keşfedeceksin.

Bu üçüncü kısımda, bağlı listeyi (linked list) manipüle etmek için bir yapı (structure) kullanan fonksiyonlar implemente etmelisin. Bunu yapmak için, `libft.h` dosyana şu yapı tanımını ekle:

```
typedef struct s_list
{
    void *content;
    struct s_list *next;
} t_list;
```

`t_list` struct'ının üyeleri:

- `content` : node içinde bulunan veri. `void *` kullanmak herhangi bir tipte veri saklamana olanak tanır.
- `next` : bir sonraki node'un adresi, veya mevcut node sonuncuysa `NULL`.

Listelerini kolayca kullanmak için şu fonksiyonları implemente et:

ft_lstnew

Prototip	<code>t_list *ft_lstnew(void *content);</code>
-----------------	--

Parametreler	<code>content</code> : yeni node'da saklanacak içerik.
Dönüş Değeri	Yeni node'a bir pointer.
External Function	<code>malloc</code>
Açıklama	Bellek tahsis eder (<code>malloc(3)</code> kullanarak) ve yeni bir node döndürür. <code>content</code> üye değişkeni verilen <code>content</code> parametresiyle initialize edilir. <code>next</code> değişkeni <code>NULL</code> olarak initialize edilir.

ft_lstadd_front

Prototip	<code>void ft_lstadd_front(t_list **lst, t_list *new);</code>
Parametreler	<code>lst</code> : bir listenin ilk node'una pointer'ın adresi. <code>new</code> : eklenecek node'a pointer'ın adresi.
Dönüş Değeri	Yok.
External Function	Yok.
Açıklama	<code>new</code> node'unu listenin başına ekler.

ft_lstsize

Prototip	<code>unsigned int ft_lstsize(t_list *lst);</code>
Parametreler	<code>lst</code> : listenin başlangıcı.
Dönüş Değeri	Listenin uzunluğu.
External Function	Yok.
Açıklama	Listedeki node sayısını sayar.

ft_lstlast

Prototip	<code>t_list *ft_lstlast(t_list *lst);</code>
-----------------	---

Parametreler	<code>lst</code> : listenin başlangıcı.
Dönüş Değeri	Listenin son node'u.
External Function	Yok.
Açıklama	Listenin son node'unu döndürür.

ft_lstadd_back

Prototip	<code>void ft_lstadd_back(t_list **lst, t_list *new);</code>
Parametreler	<code>lst</code> : bir listenin ilk node'una pointer'ın adresi. <code>new</code> : eklenecek node'a pointer'ın adresi.
Dönüş Değeri	Yok.
External Function	Yok.
Açıklama	<code>new</code> node'unu listenin sonuna ekler.

ft_lstdelone

Prototip	<code>void ft_lstdelone(t_list *lst, void (*del)(void *));</code>
Parametreler	<code>lst</code> : serbest bırakılacak node. <code>del</code> : içeriği silmek için kullanılan fonksiyonun adresi.
Dönüş Değeri	Yok.
External Function	<code>free</code>
Açıklama	Bir node'u parametre olarak alır ve içeriğini <code>del</code> fonksiyonunu kullanarak serbest bırakır. Node'un kendisini serbest bırakır ama bir sonraki node'u serbest BIRAKMAZ.

ft_lstclear

Prototip	<code>void ft_lstclear(t_list **lst, void (*del)(void *));</code>
-----------------	---

Parametreler	<code>lst</code> : bir node'a pointer'ın adresi. <code>del</code> : node'un içeriğini silmek için kullanılan fonksiyonun adresi.
Dönüş Değeri	Yok.
External Function	<code>free</code>
Açıklama	Verilen node'u ve tüm ardılarını (successors), <code>del</code> fonksiyonunu ve <code>free(3)</code> 'ü kullanarak siler ve serbest bırakır. Son olarak, listeye olan pointer'ı <code>NULL</code> olarak ayarlar.

ft_lstiter

Prototip	<code>void ft_lstiter(t_list *lst, void (*f)(void *));</code>
Parametreler	<code>lst</code> : bir node'a pointer'ın adresi. <code>f</code> : her node'un içeriğine uygulanacak fonksiyonun adresi.
Dönüş Değeri	Yok.
External Function	Yok.
Açıklama	<code>lst</code> listesinde iterasyon yapar ve <code>f</code> fonksiyonunu her node'un içeriğine uygular.

ft_lstmap

Prototip	<code>t_list *ft_lstmap(t_list *lst, void *(*f)(void *), void (*del)(void *));</code>
Parametreler	<code>lst</code> : bir node'a pointer'ın adresi. <code>f</code> : her node'un içeriğine uygulanan fonksiyonun adresi. <code>del</code> : gerekirse bir node'un içeriğini silmek için kullanılan fonksiyonun adresi.
Dönüş Değeri	Yeni liste. Tahsis başarısız olursa <code>NULL</code> .
External Function	<code>malloc</code> , <code>free</code>

Açıklama

`lst` listesinde iterasyon yapar, `f` fonksiyonunu her node'un içeriğine uygular ve `f` fonksiyonunun ardışık uygulamalarının sonucunda yeni bir liste oluşturur. `del` fonksiyonu, gerekirse bir node'un içeriğini silmek için kullanılır.

Bölüm V — Readme Gereksinimleri

Git deponun kök dizininde bir `README.md` dosyası bulunmalıdır. Amacı, projeye tanışık olmayan herkesin (akranlar, staff, işe alım uzmanları vb.) projenin ne hakkında olduğunu, nasıl çalıştırılacağını ve konu hakkında daha fazla bilgiye nerede ulaşılacağını hızlıca anlamasını sağlamaktır.

`README.md` en azından şunları içermelidir:

- İlk satır italik olmalı ve şöyle yazmalıdır: *This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]]*.
- Projeyi hedefi ve kısa bir genel bakışla açıkça sunan bir "**Description**" (Açıklama) bölümü.
- Derleme, kurulum ve/veya çalıştırma hakkında ilgili bilgileri içeren bir "**Instructions**" (Talimatlar) bölümü.
- Konuyla ilgili klasik referansları (dokümantasyon, makaleler, tutoriallar vb.) listeleyen ve AI'nın nasıl kullanıldığını — hangi görevler ve projenin hangi kısımları için — belirten bir "**Resources**" (Kaynaklar) bölümü.

➔ **Projeye bağlı olarak ek bölümler gerekebilir** (örn. kullanım örnekleri, özellik listesi, teknik seçimler vb.).

Gerekli olan herhangi bir ek, aşağıda açıkça listelenecektir.

- Bu proje için oluşturulan kütüphanenin detaylı bir açıklaması da eklenmelidir.

İngilizce tavsiye edilir; alternatif olarak, kampüsünün ana dilini kullanabilirsin.

Bölüm VI — Submission and peer-evaluation (Teslim ve Akran Değerlendirmesi)

Ödevini her zamanki gibi Git deponda teslim et. Akran değerlendirme sırasında yalnızca deponun içindeki çalışma değerlendirilecektir. Dosyalarının isimlerinin doğru olduğundan emin olmak için iki kez kontrol et.

⚠️ **Tüm dosyalarını deponun kök dizinine yerleştir.**

Değerlendirme sırasında, projede zaman zaman kısa bir modifikasyon istenebilir. Bu, küçük bir davranış değişikliği, yazılacak veya yeniden yazılacak birkaç satır kod, ya da kolayca eklenebilecek bir özellik içerebilir.

Bu adım her projeye uygulanabilir olmasa da, değerlendirme kurallarında belirtilmişse buna hazırlıklı olmalısın.

Bu adım, projenin belirli bir kısmı hakkındaki gerçek anlayışını doğrulamak içindir. Modifikasyon, seçtiğin herhangi bir geliştirme ortamında (örn. her zamanki kurulumun) gerçekleştirilebilir ve değerlendirmenin bir parçası olarak belirli bir zaman dilimi tanımlanmadıkça birkaç dakika içinde yapılabilir olmalıdır.

Örneğin, bir fonksiyonda veya scriptte küçük bir güncelleme yapman, bir görüntüyü (display) değiştirmen ya da yeni bilgi saklamak için bir veri yapısını (data structure) ayarlaman istenebilir.

Detaylar (kapsam, hedef vb.) **evaluation guidelines**'da (değerlendirme kuralları) belirtilecektir ve aynı proje için değerlendirmeden değerlendirmeye farklılık gösterebilir.